

DevOps Complete Guide

Basic to Advanced: concepts, commands and code snippets

Linux | Git | Cloud (AWS, Azure, GCP) | Docker | Kubernetes | Terraform | Ansible | Jenkins | CI/CD | Helm |
Monitoring | GitOps | DevSecOps

Each chapter goes from first principles to advanced usage, with comparison tables, ready-to-run snippets, common mistakes and interview-style questions.

Contents

5. Docker and Containers	3
5.1 Concepts	3
5.2 Essential commands	3
5.3 Dockerfile	4
5.4 Networking	5
5.5 Storage	5
5.6 Docker Compose	6
5.7 Security and troubleshooting	6
 6. Kubernetes	 8
6.1 Why Kubernetes?	8
6.2 Architecture	8
6.3 Core objects	8
6.4 kubectl essentials	9
6.5 Workload manifests	9
6.6 Configuration, secrets and storage	10
6.7 StatefulSet, Jobs and DaemonSet snippets	11
6.8 Scheduling controls	11
6.9 Autoscaling	11
6.10 Security: RBAC and policies	11
6.11 Helm: the package manager	12
6.12 Deployment strategies	12
6.13 Networking and DNS	12
6.14 Troubleshooting	12

5. Docker and Containers

5.1 Concepts

A **container** is an isolated process (or group of processes) that packages an application with its libraries and dependencies but **shares the host OS kernel**. It is built from a read-only **image** and starts in milliseconds.

Linux features that make containers possible:

- **Namespaces**: isolate what a process can *see* (PID, network, mount, user, hostname).
- **cgroups**: limit what a process can *use* (CPU, memory, IO).
- **Union filesystems (overlays)**: stack read-only image layers plus one thin writable layer.

Container vs virtual machine

	Container	Virtual machine
Isolation	Process level, shared kernel	Full OS per VM via hypervisor
Size	MBs	GBs
Startup	Milliseconds to seconds	Tens of seconds to minutes
Density	Hundreds per host	Few per host
Security boundary	Weaker (kernel shared)	Stronger
Best for	Microservices, CI, scaling	Different OS kernels, strong isolation

Docker architecture

- **Docker client** (`docker` CLI) talks to the **Docker daemon** (`dockerd`) via a REST API/socket (`/var/run/docker.sock`). The daemon uses **containerd** and **runc** to run containers.
- **Image**: immutable template. **Container**: running instance of an image. **Registry**: storage for images (Docker Hub, ECR, GHCR, Harbor). **Tag**: a label like `nginx:1.27`; **digest** (`sha256:...`) pins an exact image.
- Standards: OCI (Open Container Initiative) image and runtime specs mean Podman, containerd and CRI-O are interchangeable in many cases.

5.2 Essential commands

```
docker version; docker info
docker pull nginx:1.27-alpine
docker run -d --name web -p 8080:80 nginx:1.27-alpine      # detached, map host 8080 -> container 80
docker run -it --rm ubuntu:24.04 bash                    # interactive shell, auto remove on exit
docker run -d -e DB_HOST=db -v pgdata:/var/lib/postgresql/data --memory=512m --cpus=1 postgres:16
docker ps; docker ps -a                                  # running; all including stopped
docker logs -f --tail 100 web                             # follow logs
docker exec -it web sh                                    # shell into running container
docker inspect web | jq '.[0].NetworkSettings.IPAddress'
docker stats                                              # live CPU/mem per container
docker stop web; docker start web; docker restart web; docker rm -f web
docker images; docker rmi nginx:1.27-alpine
docker cp web:/etc/nginx/nginx.conf ./nginx.conf         # copy files in/out
docker system df; docker system prune -af --volumes      # reclaim disk (careful!)
docker login; docker tag myapp:1.0 myrepo/myapp:1.0; docker push myrepo/myapp:1.0
```

`-p HOST:CONTAINER` publishes a port. `-v` mounts storage. `-e/--env-file` pass configuration. `--restart unless-stopped` makes containers survive reboots. Containers are **ephemeral**: data written inside a container is lost when it is removed unless stored in a volume.

5.3 Dockerfile

A Dockerfile is the recipe for building an image. Each instruction creates a **layer**; unchanged layers are cached.

Instruction	Purpose
FROM	Base image (start of each stage)
WORKDIR	Set working directory
COPY / ADD	Copy files in (ADD also extracts tarballs and fetches URLs; prefer COPY)
RUN	Execute a command at build time (install packages, compile)
ENV / ARG	Runtime environment variable / build-time variable
EXPOSE	Document the listening port (does not publish it)
USER	Run as non-root user
CMD	Default command (easily overridden at docker run)
ENTRYPOINT	Fixed executable; CMD supplies default arguments to it
HEALTHCHECK	Command Docker runs to test container health
VOLUME	Declare mount point for persistent data

Example: Node.js app with **multi-stage build** and best practices.

```
# ---- build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci                # deps first: this layer is cached until package.json changes
COPY . .
RUN npm run build

# ---- runtime stage (small, no build tools) ----
FROM node:20-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
USER node                # do not run as root
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

Java Spring Boot example:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:21-jre-alpine
COPY --from=build /src/target/*.jar /app/app.jar
USER 1001
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app/app.jar"]
```

Build and run:

```
docker build -t myapp:1.0 .
docker build -t myapp:1.0 -f Dockerfile.prod --build-arg VERSION=1.0 .
docker buildx build --platform linux/amd64,linux/arm64 -t me/myapp:1.0 --push . # multi-arch
```

.dockerignore (like **.gitignore**) keeps **node_modules**, **.git**, ***.log**, **.env** out of the build context so builds are fast and secrets do not leak.

CMD vs ENTRYPOINT, shell vs exec form

- **Exec form CMD** `["node", "app.js"]` runs the process directly as PID 1 and receives signals (graceful shutdown works). **Shell form CMD** `node app.js` wraps in `/bin/sh -c` and often swallows SIGTERM.
- **ENTRYPOINT** `["java", "-jar", "app.jar"]` + **CMD** `["--spring.profiles.active=dev"]` -> arguments can be overridden at run time while the executable stays fixed.

Image optimization and best practices

- 1 Use small, specific base images (`-alpine`, `-slim`, distroless) and **pin versions**, never rely on `latest`.
- 2 Order instructions from least to most frequently changing to maximize layer caching.
- 3 Use **multi-stage builds** to keep compilers and tests out of the final image.
- 4 Run as **non-root**; drop capabilities; use read-only root filesystem where possible.
- 5 One main process per container; log to stdout/stderr so the platform collects logs.
- 6 Combine related **RUN** steps and clean caches (`apt-get clean && rm -rf /var/lib/apt/lists/*`).
- 7 Scan images (Trivy, Gype, Docker Scout) and sign them (cosign).
- 8 Never bake secrets into images; inject at runtime.

5.4 Networking

Driver	Behavior
bridge (default)	Private network on the host; containers talk by IP; user-defined bridges add DNS by container name
host	Container shares the host network stack (no port mapping, fastest)
none	No networking
overlay	Multi-host network (Docker Swarm)
macvlan	Container gets its own MAC/IP on the physical LAN

```
docker network create appnet
docker run -d --name db --network appnet postgres:16
docker run -d --name api --network appnet -e DB_HOST=db myapi:1.0 # reach DB at hostname "db"
docker network ls; docker network inspect appnet
```

5.5 Storage

Type	Command	Use
Volume (managed by Docker)	<code>-v pgdata:/var/lib/postgresql/data</code>	Databases, persistent data; preferred
Bind mount	<code>-v \$(pwd)/src:/app/src</code>	Live code reload in development
tmpfs	<code>--tmpfs /tmp</code>	In-memory temp data, secrets

```
docker volume create pgdata; docker volume ls; docker volume inspect pgdata
docker run --rm -v pgdata:/data -v $(pwd):/backup alpine tar czf /backup/pg.tgz -C /data . # backup
```

5.6 Docker Compose

Compose defines and runs multi-container apps from one YAML file.

```
# compose.yaml
services:
  api:
    build: ./api
    ports: ["8080:8080"]
    environment:
      DB_HOST: db
      DB_PASSWORD_FILE: /run/secrets/db_pass
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    secrets: [db_pass]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_pass
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5
  cache:
    image: redis:7-alpine
volumes:
  pgdata:
secrets:
  db_pass:
    file: ./db_pass.txt
```

```
docker compose up -d --build      # build and start in background
docker compose ps; docker compose logs -f api
docker compose exec db psql -U postgres
docker compose down -v           # stop and remove containers, networks and volumes
```

Compose creates a default network so services reach each other by service name (**db**, **cache**). Use profiles and override files (**compose.override.yaml**) for dev vs prod differences.

5.7 Security and troubleshooting

- Do not mount **/var/run/docker.sock** into containers (gives root-equivalent on the host). Do not use **--privileged** unless unavoidable.
- Limit resources (**--memory**, **--cpus**, **--pids-limit**), use **--read-only**, **--cap-drop ALL**, **--security-opt no-new-privileges**.
- **Rootless Docker/Podman** reduce the blast radius of a compromise.

Problem	Check
Container exits immediately	docker logs , docker inspect --format '{{.State.ExitCode}}' ; exit 137 = killed (often OOM), 1 = app error
Cannot reach app	Port mapping, app binding to 0.0.0.0 (not 127.0.0.1), firewall
Disk fills up	docker system df , prune unused images/volumes, log rotation (--log-opt max-size=10m)
Build slow	Layer order, .dockerignore , BuildKit cache mounts

Docker vs Podman vs containerd

Docker = full developer experience with a daemon. **Podman** is daemonless and rootless by default and CLI-compatible (**alias** `docker=podman`). **containerd/CRI-O** are lean runtimes used by Kubernetes (Kubernetes removed the Docker shim in 1.24, but Docker-built images still run perfectly).

Interview questions

- Image vs container? Image is the read-only template; container is a running instance with a writable layer.
- COPY vs ADD? COPY copies files only; ADD also extracts archives and downloads URLs.
- How do you reduce image size? Multi-stage builds, minimal base images, clean caches, `.dockerignore`.
- How does data persist? Volumes or bind mounts, external storage.
- What is a dangling image? An untagged layer left from a rebuild; remove with `docker image prune`.

6. Kubernetes

6.1 Why Kubernetes?

Running one container is easy; running hundreds across many machines needs **orchestration**: scheduling, self-healing, scaling, service discovery, rolling updates, configuration and secrets. Kubernetes (K8s) is the open-source standard for that. You declare the **desired state** in YAML; controllers continuously reconcile **actual state** toward it.

6.2 Architecture

Control plane (the brain):

- **kube-apiserver**: front door; every component and **kubectl** talks to it (REST).
- **etcd**: consistent key-value store holding all cluster state (back it up!).
- **kube-scheduler**: places pods on nodes based on resources, affinity, taints.
- **kube-controller-manager**: runs controllers (Deployment, ReplicaSet, Node, Job...) that reconcile state.
- **cloud-controller-manager**: integrates with cloud load balancers, volumes, nodes.

Worker node:

- **kubelet**: agent that makes sure the pod's containers run and report health.
- **kube-proxy**: programs networking rules (iptables/IPVS) for Services.
- **Container runtime**: containerd or CRI-O.

Managed offerings (EKS, AKS, GKE) run the control plane for you. Local learning: **minikube**, **kind**, **k3d**, Docker Desktop.

6.3 Core objects

Object	Purpose
Pod	Smallest deployable unit: one or more containers sharing network (one IP) and volumes. Ephemeral.
ReplicaSet	Keeps N identical pods running
Deployment	Declarative updates for stateless apps: manages ReplicaSets, rolling updates and rollbacks
StatefulSet	Pods with stable identity (db-0 , db-1), stable storage; for databases and queues
DaemonSet	One pod on every (or selected) node: log collectors, monitoring agents
Job / CronJob	Run-to-completion tasks / scheduled tasks
Service	Stable virtual IP and DNS name that load-balances to matching pods
Ingress	HTTP(S) routing (host/path) into the cluster, needs an Ingress controller (NGINX, Traefik)
ConfigMap / Secret	Configuration / sensitive data injected as env vars or files
PersistentVolume (PV) / PVC	Storage resource / a request for storage; StorageClass provisions dynamically
Namespace	Virtual cluster for isolation, quotas and RBAC
HPA / VPA	Horizontal / vertical autoscaling of pods
ServiceAccount, Role, RoleBinding	Identity and RBAC permissions
NetworkPolicy	Firewall rules between pods

6.4 kubectl essentials

```

kubectl config get-contexts; kubectl config use-context prod; kubectl config set-context --current --nam \
  espace=dev
kubectl get nodes -o wide
kubectl get pods -A # all namespaces
kubectl get pods -n dev -l app=web -o wide --watch
kubectl describe pod web-abc123 # events: great for troubleshooting
kubectl logs web-abc123 -c app -f --previous # --previous = logs of crashed container
kubectl exec -it web-abc123 -- sh
kubectl apply -f deploy.yaml # create or update declaratively
kubectl delete -f deploy.yaml
kubectl create deployment web --image=nginx --replicas=3 --dry-run=client -o yaml > deploy.yaml
kubectl scale deployment web --replicas=5
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web; kubectl rollout history deployment/web
kubectl rollout undo deployment/web # rollback
kubectl port-forward svc/web 8080:80 # test a service locally
kubectl top pods; kubectl top nodes # needs metrics-server
kubectl get events --sort-by=.lastTimestamp
kubectl explain deployment.spec.strategy # built-in docs for any field
kubectl auth can-i create pods --as=system:serviceaccount:dev:ci

```

6.5 Workload manifests

Deployment + Service

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: dev
  labels: { app: web }
spec:
  replicas: 3
  selector:
    matchLabels: { app: web } # must match template labels
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: myrepo/web:1.4.2
          ports: [{ containerPort: 8080 }]
          env:
            - name: DB_HOST
              valueFrom: { configMapKeyRef: { name: web-config, key: db_host } }
            - name: DB_PASSWORD
              valueFrom: { secretKeyRef: { name: web-secret, key: password } }
          resources:
            requests: { cpu: 100m, memory: 128Mi } # used for scheduling
            limits: { cpu: 500m, memory: 256Mi } # hard ceiling
          readinessProbe: # gets traffic only when ready
            httpGet: { path: /health/ready, port: 8080 }
            initialDelaySeconds: 5
          livenessProbe: # restarts container when unhealthy
            httpGet: { path: /health/live, port: 8080 }
            initialDelaySeconds: 15
---
apiVersion: v1
kind: Service
metadata: { name: web, namespace: dev }
spec:
  type: ClusterIP
  selector: { app: web }
  ports: [{ port: 80, targetPort: 8080 }]

```

Service types

Type	Behavior
ClusterIP (default)	Internal-only virtual IP
NodePort	Opens a port (30000-32767) on every node
LoadBalancer	Provisions a cloud load balancer (AWS ELB, Azure LB)
ExternalName	DNS CNAME alias to an external hostname
Headless (clusterIP: None)	No virtual IP; DNS returns pod IPs (used by StatefulSets)

Ingress

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations: { cert-manager.io/cluster-issuer: letsencrypt }
spec:
  ingressClassName: nginx
  tls: [{ hosts: [app.example.com], secretName: web-tls }]
  rules:
    - host: app.example.com
      http:
        paths:
          - { path: /api, pathType: Prefix, backend: { service: { name: api, port: { number: 80 } } } }
          - { path: /, pathType: Prefix, backend: { service: { name: web, port: { number: 80 } } } }
```

6.6 Configuration, secrets and storage

```

apiVersion: v1
kind: ConfigMap
metadata: { name: web-config }
data:
  db_host: db.dev.svc.cluster.local
  app.properties: |
    log.level=info
---
apiVersion: v1
kind: Secret
metadata: { name: web-secret }
type: Opaque
stringData: { password: "s3cr3t" }      # stored base64-encoded (NOT encrypted by default)
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources: { requests: { storage: 10Gi } }
```

Kubernetes Secrets are only base64 encoded. Enable encryption at rest for etcd, restrict RBAC, and prefer external secret managers (External Secrets Operator, Vault, cloud KMS/Secrets Manager).

Access modes: **RWO** (one node read-write), **ROX** (many nodes read-only), **RWX** (many nodes read-write, needs NFS/EFS-type storage).

6.7 StatefulSet, Jobs and DaemonSet snippets

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: nightly-backup }
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - { name: backup, image: myrepo/backup:1.0, args: ["--target", "s3://bkp"] }
```

StatefulSet uses `serviceName: <headless-svc>` and `volumeClaimTemplates` so every replica gets its own PVC that survives rescheduling.

6.8 Scheduling controls

- **nodeSelector / node affinity:** run only on nodes with certain labels. **Pod anti-affinity:** spread replicas across nodes or zones.
- **Taints and tolerations:** a taint repels pods from a node (`kubectl taint nodes gpu1 gpu=true:NoSchedule`); only pods with a matching toleration may land there.
- **Topology spread constraints:** even distribution across zones.
- **PodDisruptionBudget:** guarantees a minimum number of pods stay up during voluntary disruptions like node drains.
- **PriorityClass:** important pods may preempt less important ones.

6.9 Autoscaling

Scaler	What it scales	Based on
HPA	Number of pod replicas	CPU, memory, custom metrics
VPA	Pod CPU/memory requests	Observed usage
Cluster Autoscaler / Karpenter	Number of nodes	Pending pods

```
kubectl autoscale deployment web --cpu-percent=70 --min=3 --max=10
```

HPA needs **resource requests** defined and metrics-server installed.

6.10 Security: RBAC and policies

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: read-pods, namespace: dev }
subjects: [{ kind: User, name: alice, apiGroup: rbac.authorization.k8s.io }]
roleRef: { kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io }
```

Role/RoleBinding are namespaced; **ClusterRole/ClusterRoleBinding** are cluster-wide. Apply **NetworkPolicies** (default deny, then allow), Pod Security Standards (restricted), non-root users, read-only filesystems, image scanning

and admission policies (OPA Gatekeeper, Kyverno).

6.11 Helm: the package manager

Helm packages manifests as a **chart** with templated YAML and a **values.yaml**; an installed chart is a **release**.

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
helm search repo nginx
helm install web bitnami/nginx -n dev --create-namespace -f my-values.yaml --set replicaCount=3
helm upgrade --install web ./mychart -n dev -f values-prod.yaml
helm list -A; helm history web; helm rollback web 1; helm uninstall web
helm template ./mychart | kubectl apply --dry-run=client -f - # render locally
helm create mychart # scaffold a chart
```

Template snippet: `image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"`. **Kustomize** (built into kubectl via `kubectl apply -k`) is the template-free alternative using base + overlays with patches.

6.12 Deployment strategies

Strategy	How	Trade-off
Rolling update (default)	Replace pods gradually	Zero downtime, both versions run briefly
Recreate	Stop all, then start new	Downtime; simple, avoids version mix
Blue/Green	Two full environments, switch Service selector or LB	Instant rollback, double resources
Canary	Send small % of traffic to new version	Low risk; needs ingress/service mesh (Argo Rollouts, Istio)

6.13 Networking and DNS

Every pod gets its own IP (flat network via a **CNI plugin**: Calico, Cilium, Flannel, AWS VPC CNI). Service DNS: `<service>.<namespace>.svc.cluster.local`; pods in the same namespace can simply use `<service>`. kube-proxy and CoreDNS provide load balancing and name resolution.

6.14 Troubleshooting

Status	Meaning and fix
Pending	Not scheduled: insufficient resources, unbound PVC, taints. <code>kubectl describe pod</code> Events
ImagePullBackOff / ErrImagePull	Wrong image name/tag, or missing <code>imagePullSecrets</code> for private registry
CrashLoopBackOff	App keeps crashing: <code>kubectl logs --previous</code> , wrong command/config, failing probes
OOMKilled	Exceeded memory limit: raise limit or fix leak
CreateContainerConfigError	Missing ConfigMap/Secret key
Service has no endpoints	Selector does not match pod labels; pods not Ready. <code>kubectl get endpoints svc</code>
Evicted	Node under pressure (disk/memory)

Debug flow: `get -> describe -> logs -> exec / kubectl debug -> check Events, Service endpoints, DNS (nslookup from a debug pod), NetworkPolicy`.

Kubernetes vs Docker Swarm vs ECS

	Kubernetes	Docker Swarm	AWS ECS
Complexity	High, very powerful	Low	Medium
Ecosystem	Largest (Helm, operators, service meshes)	Small	AWS-integrated
Portability	Any cloud or on-prem	Any	AWS only
Best for	Production at scale	Simple small clusters	Teams all-in on AWS

Interview questions

- Pod vs container? Pod wraps one or more containers sharing network and storage.
- Deployment vs StatefulSet? Interchangeable stateless replicas vs stable identity and storage.
- Liveness vs readiness probe? Liveness restarts unhealthy containers; readiness removes pods from Service endpoints until ready. A **startup probe** protects slow-starting apps.
- ClusterIP vs NodePort vs LoadBalancer vs Ingress? Internal; node port; cloud LB per service; one HTTP entry point routing to many services.
- What happens when you run `kubectl apply`? API server authenticates, authorizes, admits and stores in etcd; controllers create ReplicaSet and Pods; scheduler binds to nodes; kubelet starts containers.
- Requests vs limits? Requests guarantee and drive scheduling; limits cap usage (CPU is throttled, memory excess is OOMKilled).